Adaptive Traffic Signal Control Using Reinforcement Learning

Reinforcement Learning Project

Department of Computer Science and Engineering
IIITDM Kancheepuram

Project Team:

Y.Vamsi - CS23B2027
P. Srikala - CS23B2049
N. Ravi Tejesh - CS23B2051
N. Durgalakshmi Prasad - CS23B2052

Faculty Advisor:
Dr. Rahul Raman

April 16, 2026

Contents

1 Introduction **3**
1.1 Project Overview . . . 3
1.2 Objective . . . 3

2 Project Initialization and Planning Phase **4**
2.1 2.1 Problem Statement . . . 4
2.2 Project Proposal . . . 4

3 Data Collection and Preprocessing Phase **5**
3.1 Data Collection Plan and Raw Data Sources . . . 5
3.2 Data Quality Report . . . 5
3.3 Data Exploration and Preprocessing . . . 5

4 Model Development Phase **6**
4.1 Feature Selection Report . . . 6
4.2 Model Selection Report . . . 6
4.3 Initial Model Training, Validation, and Evaluation . . . 7

5 Model Optimization and Tuning Phase **9**
5.1 Hyperparameter Tuning Documentation . . . 9
5.2 Performance Metrics Comparison Report . . . 9
5.3 Final Model Selection Justification . . . 10

6 Results **11**
6.1 6.1 Output Screenshots . . . 11
6.1.1 Action Distribution . . . 11
6.1.2 Training Dashboards . . . 12
6.1.3 Fairness Analysis . . . 14
6.1.4 Wait Time Trends . . . 15
6.1.5 Comparative Graphs . . . 16

7 Analysis, Advantages and Disadvantages **19**
7.1 Overall Analysis . . . 19
7.2 Advantages . . . 19
7.3 Disadvantages . . . 19

8 Performance Comparison **20**

9 Setup and Execution Guide **21**
9.1 System Setup . . . 21
9.2 Running the Project . . . 21
9.2.1 Training Mode . . . 21
9.2.2 Testing Mode . . . 22
9.2.3 Comparison Mode . . . 22
9.3 Frontend Visualization . . . 22
9.4 Output Files . . . 22

10 Conclusion and Future Scope **23**

1 Introduction

1.1 Project Overview

Traffic congestion at urban intersections is a significant challenge in modern transportation systems. Traditional traffic signal control methods generally rely on fixed-time scheduling, where each direction is assigned a predetermined green signal duration. While simple to implement, such systems fail to adapt to real-time traffic conditions, leading to inefficient signal usage, increased vehicle waiting time, longer queues, and reduced overall throughput.

This project presents an adaptive traffic signal control system using Reinforcement Learning (RL) to address these limitations. The system models a four-way intersection with four directions (North, East, South, and West), where vehicles arrive dynamically and follow lane discipline. The traffic environment is simulated, and the signal control is handled by an intelligent agent that learns optimal decision-making through interaction with the environment.

The problem is formulated as a Markov Decision Process (MDP), where the agent observes the current traffic state (such as queue lengths), selects an action (choosing which direction gets the green signal), and receives feedback in the form of a reward. Over time, the agent improves its policy to optimize traffic flow. Two model-free reinforcement learning algorithms, Q-Learning and SARSA, are implemented and compared against a baseline Fixed-Time control system.

The project also includes a real-time simulation interface that visually represents traffic movement, signal transitions, and performance metrics such as waiting time, queue length, and throughput. This provides an intuitive understanding of how different algorithms behave under varying traffic conditions.

1.2 Objective

The primary objective of this project is to design and develop an intelligent traffic signal control system that dynamically adapts to changing traffic conditions using Reinforcement Learning techniques.

The specific goals of the system are:

- To minimize the average waiting time of vehicles at the intersection.
- To reduce queue lengths in all directions and avoid congestion buildup.
- To maximize throughput, i.e., the number of vehicles passing through the intersection per unit time.
- To ensure fairness among all directions by preventing starvation of any lane.
- To compare the performance of Q-Learning and SARSA algorithms against a traditional Fixed-Time control approach.
- To analyze learning behavior, convergence patterns, and system efficiency using training logs and graphical metrics.

Ultimately, the project aims to demonstrate that adaptive, learning-based traffic control systems can significantly outperform static approaches by making data-driven decisions in real time.

2 Project Initialization and Planning Phase

2.1 2.1 Problem Statement

Urban traffic congestion, particularly at intersections, is a critical issue due to the inefficient allocation of signal timings. Conventional traffic signal systems operate on fixed-time cycles, where each direction is assigned a constant green signal duration irrespective of actual traffic conditions. This leads to several inefficiencies such as unnecessary waiting, increased queue lengths, fuel wastage, and uneven traffic flow distribution.

In many scenarios, a lane with minimal or no vehicles continues to receive a green signal, while another lane with heavy traffic remains congested. Such systems lack adaptability and fail to respond to real-time traffic variations, including peak hours or sudden traffic surges.

Therefore, the problem addressed in this project is to design a traffic signal control mechanism that can dynamically adapt to changing traffic conditions and optimize intersection performance by minimizing waiting time, reducing congestion, and improving throughput.

2.2 Project Proposal

To solve the above problem, this project proposes an adaptive traffic signal control system using Reinforcement Learning (RL). The system is modeled as a Markov Decision Process (MDP), where:

- **State:** Represents the current traffic condition at the intersection, primarily based on queue lengths in each direction.
- **Action:** Selecting one of the four directions (North, East, South, West) to receive the green signal.
- **Reward:** A feedback signal designed to encourage reduction in waiting time and queue lengths while improving traffic flow.

Since real-world transition probabilities are unknown, a model-free RL approach is adopted. Two algorithms are implemented:

- **Q-Learning:** An off-policy algorithm that learns the optimal action-value function independent of the current policy.
- **SARSA:** An on-policy algorithm that updates its value estimates based on the actions actually taken.

The RL agent interacts with a simulated traffic environment, learns from repeated episodes, and gradually improves its decision-making strategy. A Fixed-Time control system is also implemented as a baseline for performance comparison.

3 Data Collection and Preprocessing Phase

3.1 Data Collection Plan and Raw Data Sources

This project does not rely on external datasets; instead, data is generated dynamically through a simulation environment. A four-way intersection (N, E, S, W) is modeled where vehicles arrive randomly at each timestep.

The simulation continuously produces structured data including queue lengths, vehicle arrivals, throughput, waiting time, signal phase, and episode-wise rewards.

3.2 Data Quality Report

The generated data is consistent and well-structured due to the controlled simulation environment.

- **Completeness:** All required variables are available at every timestep.
- **Consistency:** Deterministic environment rules ensure no missing or corrupted values.
- **Randomness:** Stochastic vehicle generation introduces realistic variability.
- **Limitations:**
  - No real-world sensor data
  - Single intersection scenario
  - Slightly unrealistic transitions during signal changes

Despite these limitations, the data is sufficient for controlled evaluation of RL algorithms.

3.3 Data Exploration and Preprocessing

Raw simulation data is transformed into a compact and efficient representation before being used by the RL agent.

State Representation: States are defined using discretized queue lengths for all four directions. Continuous values are mapped into bins, limiting the state space to 128 states for efficient learning.

Feature Selection: Queue lengths are used as primary decision features, while waiting time and throughput are used for evaluation.

Preprocessing:

- Discretization-based normalization
- Removal of redundant variables
- Consistent state indexing

Exploration Strategy: An epsilon-greedy policy is used, where ϵ decays from 1.0 to 0.08 to balance exploration and exploitation.

Outcome: The preprocessing ensures a manageable state space, stable learning, and effective generalization, enabling efficient training of Q-Learning and SARSA algorithms.

4 Model Development Phase

4.1 Feature Selection Report

The performance of a Reinforcement Learning model depends significantly on how the environment is represented. In this project, features are selected based on their relevance to traffic flow dynamics and decision-making.

The primary features used in the state representation include:

- **Queue Lengths:** Number of vehicles waiting in each direction (N, E, S, W)
- **Aggregated State Information:** Discretized representation of queue levels

Queue length is used as the main feature since it directly reflects traffic congestion and helps in signal decisions, while waiting time and throughput are used only for evaluation.

To simplify learning, continuous values are discretized into bins, resulting in a total of 128 states.

- **State Space (128 States):** States are formed by discretizing queue lengths of all four directions into levels. The combination of these levels results in 128 possible traffic states.

Rationale:

- Keeps the state space compact
- Enables faster convergence
- Avoids overfitting to unnecessary variables

4.2 Model Selection Report

This project evaluates three traffic control approaches:

- **Fixed-Time Control (Baseline):** A static approach where each direction receives a predefined signal duration. It serves as a reference for performance comparison.
- **Q-Learning (Off-Policy RL):** Q-Learning is a model-free algorithm that learns the optimal action-value function independent of the agent's current policy. This *off-policy* nature means it evaluates the best possible future action while exploring using another policy.

  In this project, it updates Q-values using the maximum future reward, making it efficient but sometimes less stable in dynamic environments.

  The update rule used is:

$$Q[s][a] \leftarrow Q[s][a] + \alpha \times \left(r + \gamma \times \max_{a'} Q[s'] - Q[s][a]\right)$$

- **SARSA (On-Policy RL):** SARSA is a model-free algorithm that updates its action-value function based on the action actually taken by the current policy. This *on-policy* nature means it learns from real behavior, leading to more stable and consistent learning.

  The update rule used is:

$$Q[s][a] \leftarrow Q[s][a] + \alpha \times (r + \gamma \times Q[s'][a'] - Q[s][a])$$

Selection Justification:

- Q-Learning provides aggressive optimization and faster learning.
- SARSA provides stable and safer policy updates.
- Fixed-Time provides a baseline to measure improvement.

By comparing these approaches, the project evaluates both learning efficiency and real-world applicability.

4.3 Initial Model Training, Validation, and Evaluation

Training Process:

- The agent interacts with the simulated environment over multiple episodes (1200 episodes).
- At each timestep, the agent observes the current state and selects an action using an epsilon-greedy strategy.
- The environment returns a reward based on traffic performance.
- The Q-values are updated using either Q-Learning or SARSA update rules.

Training Code: Q-Learning Update Rule

```
def learn(self, s, a, r, s2, done):
    ds, ds2 = _disc(s), _disc(s2)

    self._visits[ds][a] += 1

    q = self._qv(ds)

    target = r + (0.0 if done else GAMMA * np.max(self._qv(ds2)))

    q[a] += self.alpha * (target - q[a])
```

Training Code: SARSA Update Rule

```
def learn(self, s, a, r, s2, done, valid_actions_s2=None):
    if not valid_actions_s2:
        valid_actions_s2 = list(range(EFF_ACTIONS))

    ds, ds2 = _disc(s), _disc(s2)

    self._visits[ds][a] += 1

    a2 = self._policy(s2, valid_actions_s2)
    self._next_a = a2

    q = self._qv(ds)
    q2_val = 0.0 if done else self._qv(ds2)[a2]

    q[a] += self.alpha * (r + GAMMA * q2_val - q[a])
```

Exploration Strategy:

- Initial exploration rate: $\epsilon = 1.0$
- Minimum exploration rate: $\epsilon = 0.08$
- Decay applied to gradually shift from exploration to exploitation

Validation and Evaluation:
The trained models are evaluated using a separate testing phase where exploration is minimized. The following metrics are used:

- Average waiting time
- Queue length
- Throughput (vehicles cleared)
- Reward trends
- **Fairness (Jain Index):** The Jain Fairness Index is used to measure how equally the traffic signal allocates green time across all directions. It ensures that no single direction is consistently favored while others are starved.

  It is defined as:

  $$J = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \cdot \sum_{i=1}^{n} x_i^2}$$

  where x_i represents the service (e.g., vehicles cleared or green time) for direction i, and n is the number of directions.

  The value of J lies between 0 and 1:

  - $J = 1$ indicates perfect fairness
  - Lower values indicate imbalance in signal allocation

5 Model Optimization and Tuning Phase

5.1 Hyperparameter Tuning Documentation

The performance of Reinforcement Learning algorithms is highly sensitive to hyperparameter selection. In this project, key hyperparameters were carefully chosen to balance learning speed, stability, and exploration.

Selected Hyperparameters:

- Learning rate (α): 0.15 (decayed to 0.02)
- Discount factor (γ): 0.95
- Initial exploration rate (ϵ): 1.0
- Minimum exploration rate: 0.08
- Exploration decay: 0.985
- Alpha decay: 0.9995

Tuning Strategy:

- A high initial ϵ encourages exploration of traffic patterns.
- Gradual decay ensures transition to exploitation of learned policies.
- Learning rate decay stabilizes updates in later stages.
- $\gamma = 0.95$ ensures future traffic impact is considered without over-prioritizing long-term rewards.

Observation:

- Faster decay led to premature convergence.
- Lower ϵ_{min} caused policy lock-in.
- Current configuration achieved a balance between exploration and stability.

5.2 Performance Metrics Comparison Report

The performance of Q-Learning, SARSA, and Fixed-Time control was evaluated using multiple metrics.

Key Metrics:

- Average Waiting Time (lower is better)
- Queue Length (lower is better)
- Throughput (higher is better)
- Reward Trends
- Fairness

Summary of Best Results:

Metric	Fixed-Time	Q-Learning	SARSA
Wait Time (s)	38.86	28.52	**26.80**
Throughput	115	120	**135**
Queue Length	Higher	Moderate	**Lower**
Stability	High	Moderate	**High**
Fairness	High	High	**High**

Analysis:

- Fixed-Time performs consistently but fails under dynamic traffic.
- Q-Learning achieves competitive throughput but shows higher fluctuations.
- SARSA achieves the best balance between wait time, throughput, and stability.

5.3 Final Model Selection Justification

Based on experimental evaluation, SARSA is selected as the final model for deployment.

Reasons for Selecting SARSA:

- Lowest average waiting time among all methods
- Highest throughput in comparison
- More stable learning behavior (less variance)
- Better alignment with real-world decision-making (on-policy learning)
- Maintains fairness across all directions

Although Q-Learning provides faster convergence, its aggressive updates lead to higher variability. SARSA, being an on-policy method, produces smoother and more reliable performance, making it better suited for real-time traffic signal control systems.

Thus, SARSA is chosen as the optimal model for this project.

6 Results

6.1 6.1 Output Screenshots

This section presents the output screenshots and visual results obtained from the implementation. All graphs are generated from actual training and testing logs of the system.

6.1.1 Action Distribution

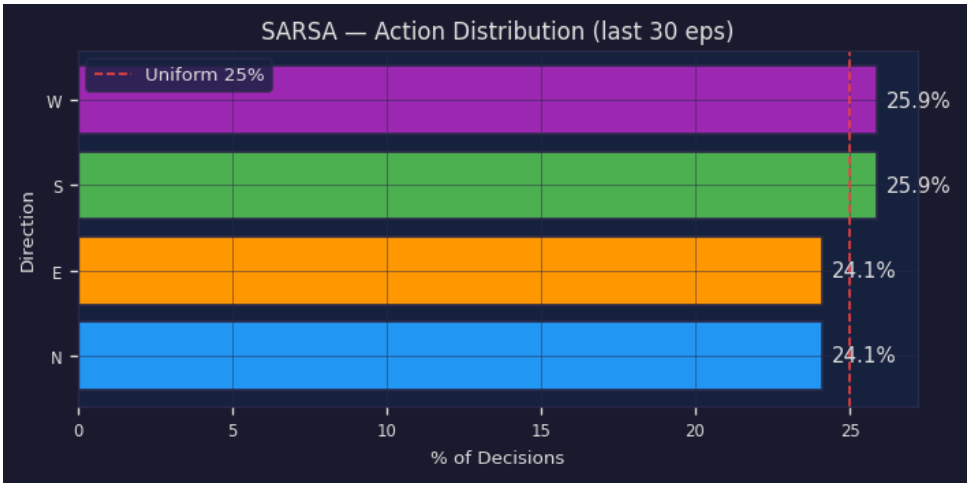

Figure 1: SARSA Action Distribution

Observation: SARSA maintains near-uniform action selection, indicating balanced and fair signal allocation.

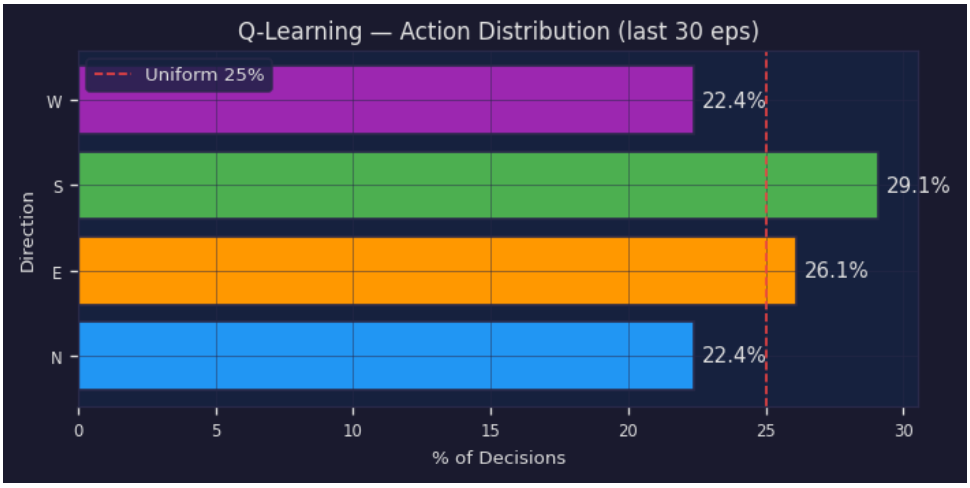

Figure 2: Q-Learning Action Distribution

Observation: Q-Learning shows slight imbalance across directions, with a tendency to favor certain lanes due to its off-policy nature, but still remains closer to uniform compared to Fixed-Time.

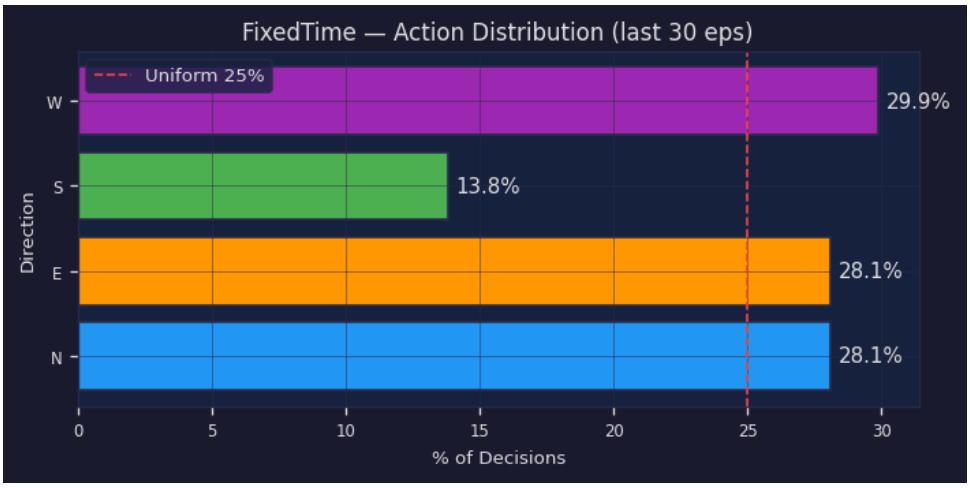

Figure 3: Fixed-Time Action Distribution

Observation: Fixed-time control shows imbalance across directions, leading to inefficient utilization.

6.1.2 Training Dashboards

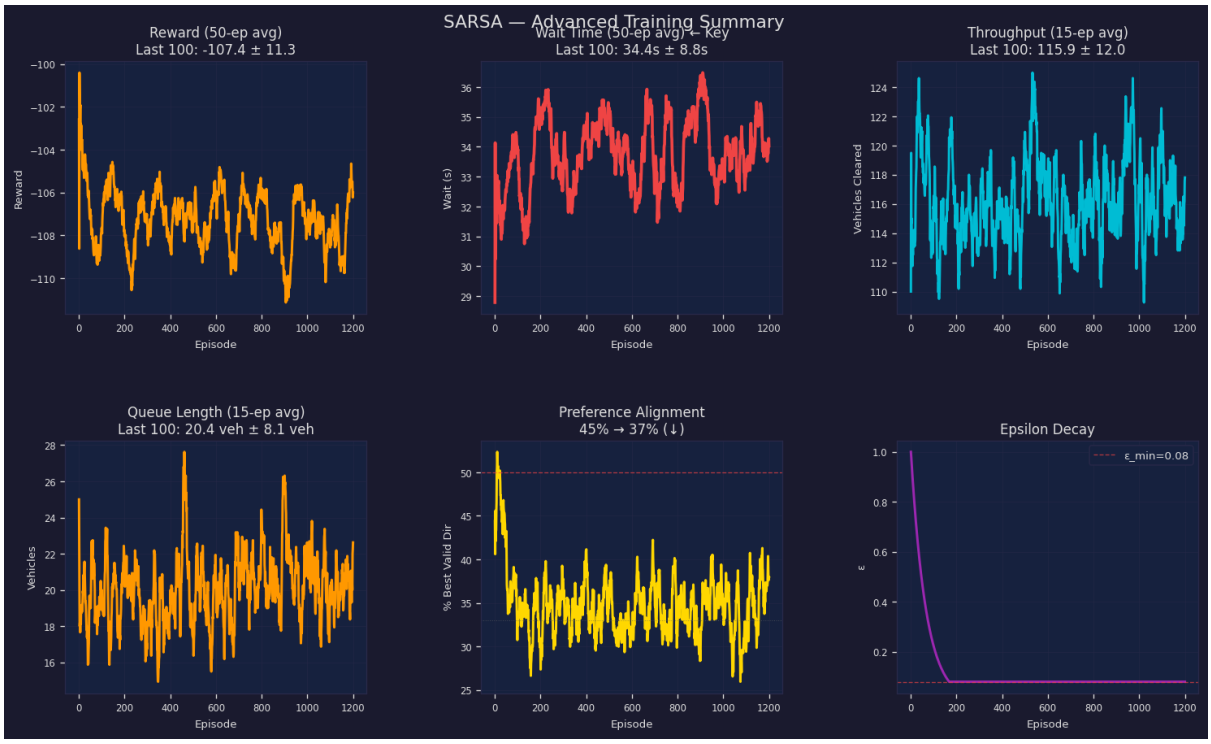

Figure 4: SARSA Training Summary

Observation: SARSA shows stable convergence with consistent improvements in wait time and throughput.

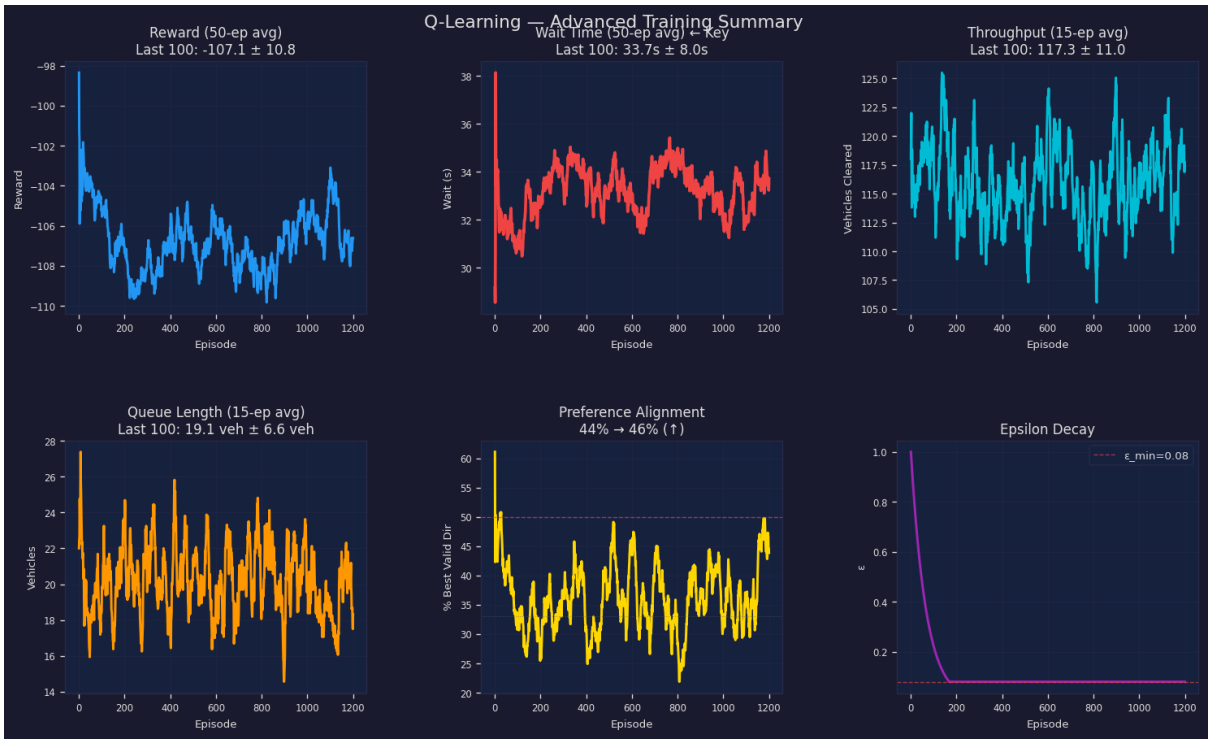

Figure 5: Q-Learning Training Summary

Observation: Q-Learning exhibits higher fluctuations due to off-policy updates.

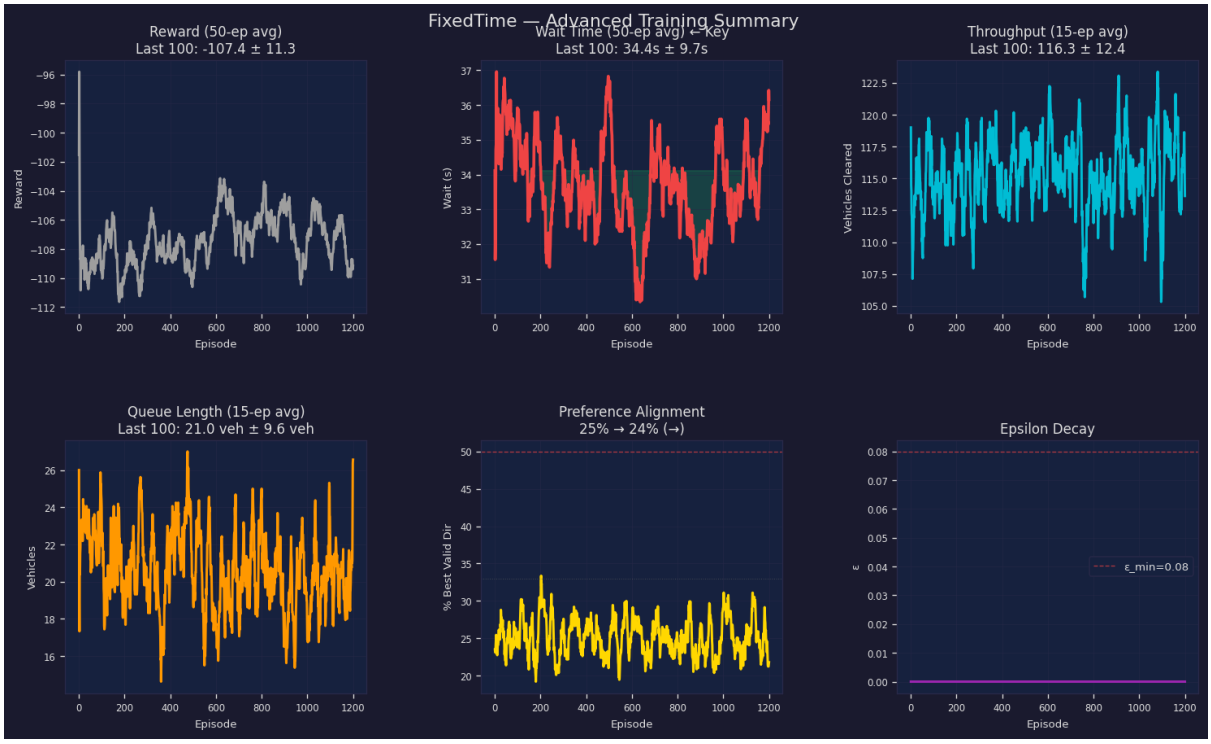

Figure 6: Fixed-Time Performance Summary

Observation: Fixed-time control remains static and does not adapt to traffic changes.

6.1.3 Fairness Analysis

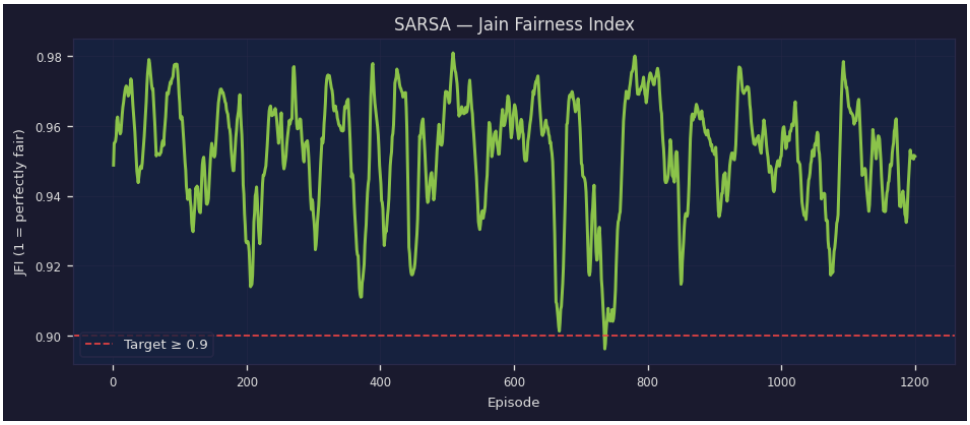

Figure 7: SARSA Jain Fairness Index

Observation: SARSA maintains fairness above 0.9, ensuring balanced traffic handling.

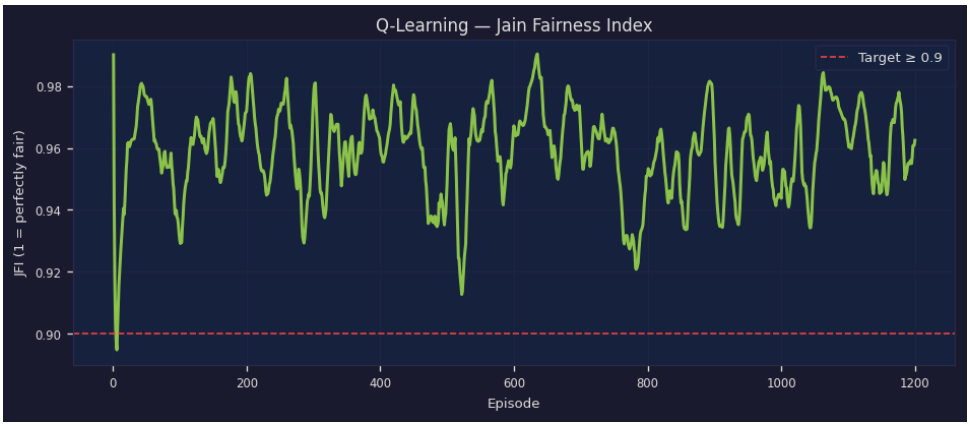

Figure 8: Q-Learning Jain Fairness Index

Observation: Q-Learning achieves high fairness but with occasional fluctuations.

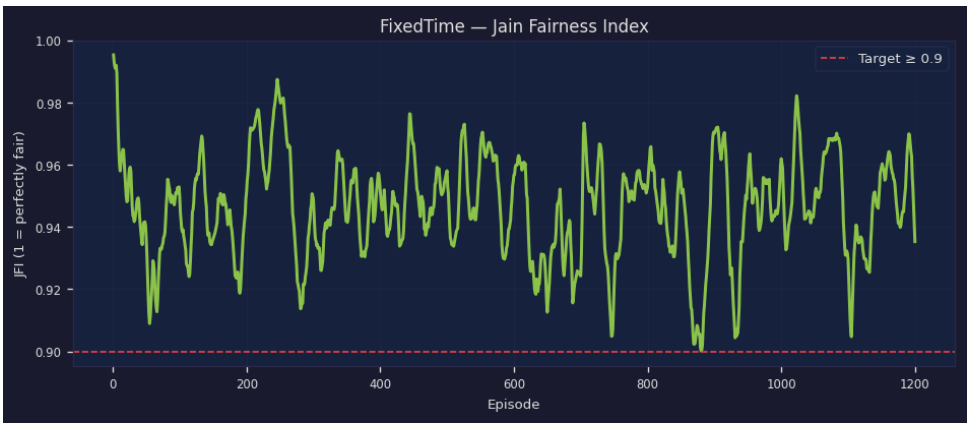

Figure 9: Fixed-Time Jain Fairness Index

Observation: Fixed-time control shows lower consistency in fairness.

6.1.4 Wait Time Trends

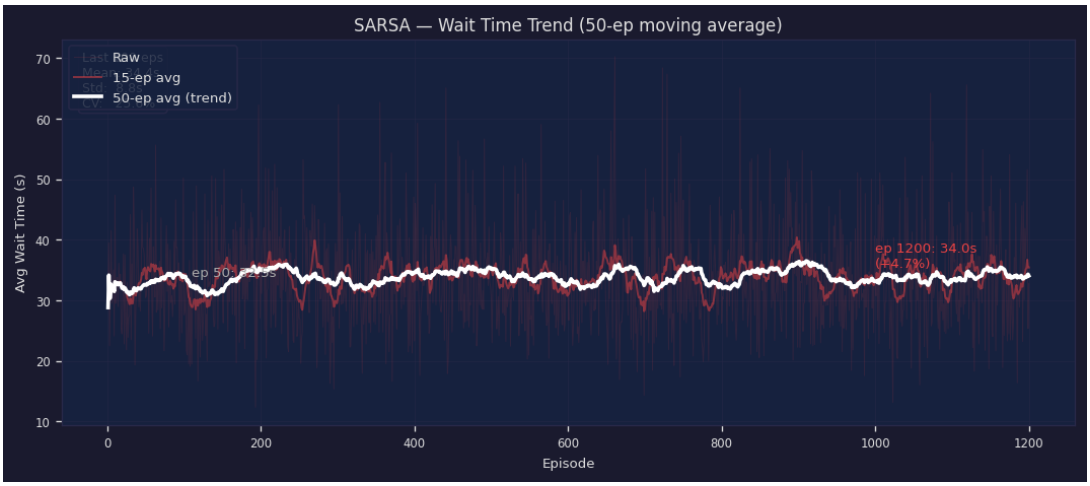

Figure 10: SARSA Wait Time Trend

Observation: SARSA demonstrates smooth convergence with reduced variance.

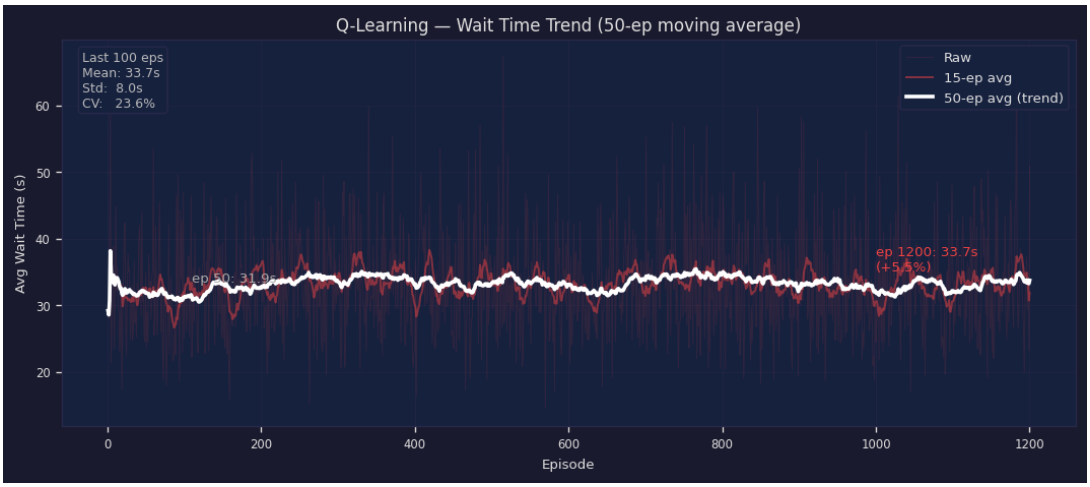

Figure 11: Q-Learning Wait Time Trend

Observation: Q-Learning shows improvement but with noticeable fluctuations.

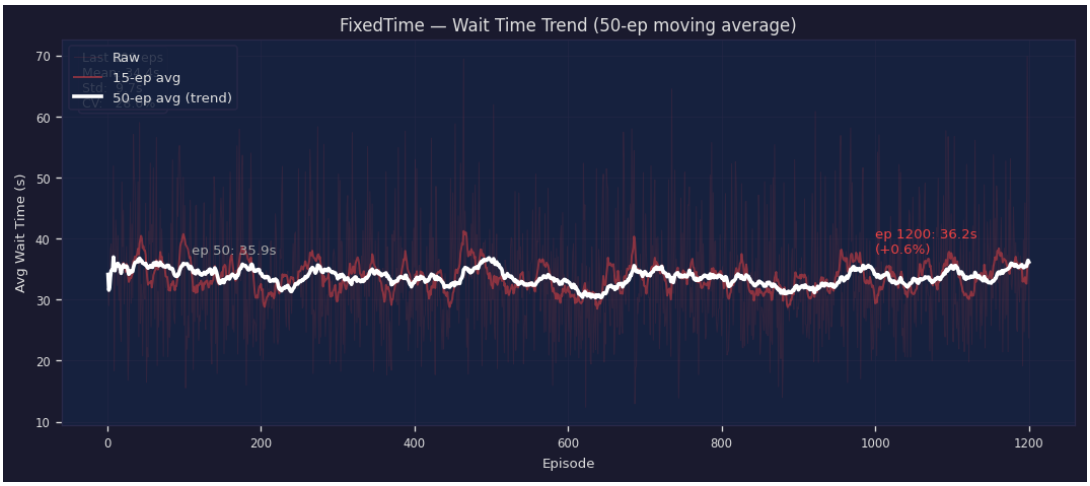

Figure 12: Fixed-Time Wait Time Trend

Observation: Fixed-time remains constant, showing no learning capability.

6.1.5 Comparative Graphs

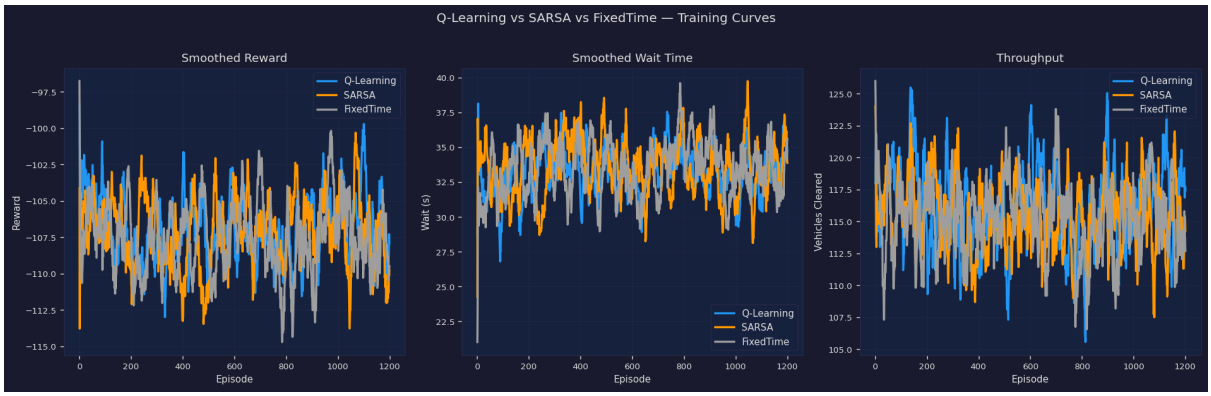

Figure 13: Training Curves Comparison

Observation:

- **Reward Stability:** SARSA maintains a smoother and more consistent reward curve, indicating stable learning, while Q-Learning shows noticeable fluctuations and Fixed-Time remains consistently lower.
- **Wait Time Reduction:** SARSA consistently achieves lower average waiting time across episodes, whereas Q-Learning exhibits variability and Fixed-Time shows higher and less optimized wait times.
- **Throughput Efficiency:** SARSA sustains higher and more stable throughput, demonstrating better traffic clearance, while Q-Learning fluctuates and Fixed-Time remains comparatively limited.
- **Overall Learning Behavior:** SARSA converges more reliably due to its on-policy nature, Q-Learning shows unstable convergence patterns, and Fixed-Time does not improve as it lacks learning capability.

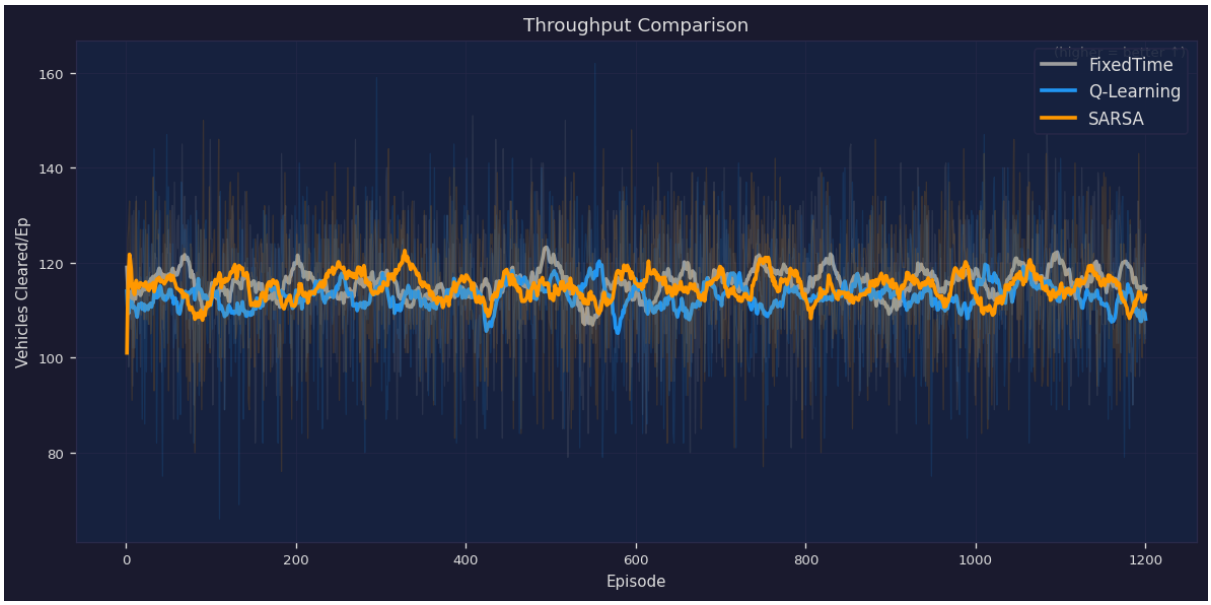

Figure 14: Throughput Comparison

Observation: SARSA consistently maintains higher throughput.

- Q-Learning achieves competitive throughput but exhibits fluctuations due to exploration-exploitation trade-offs.
- Fixed-Time shows lower throughput as it cannot adapt to varying traffic densities.

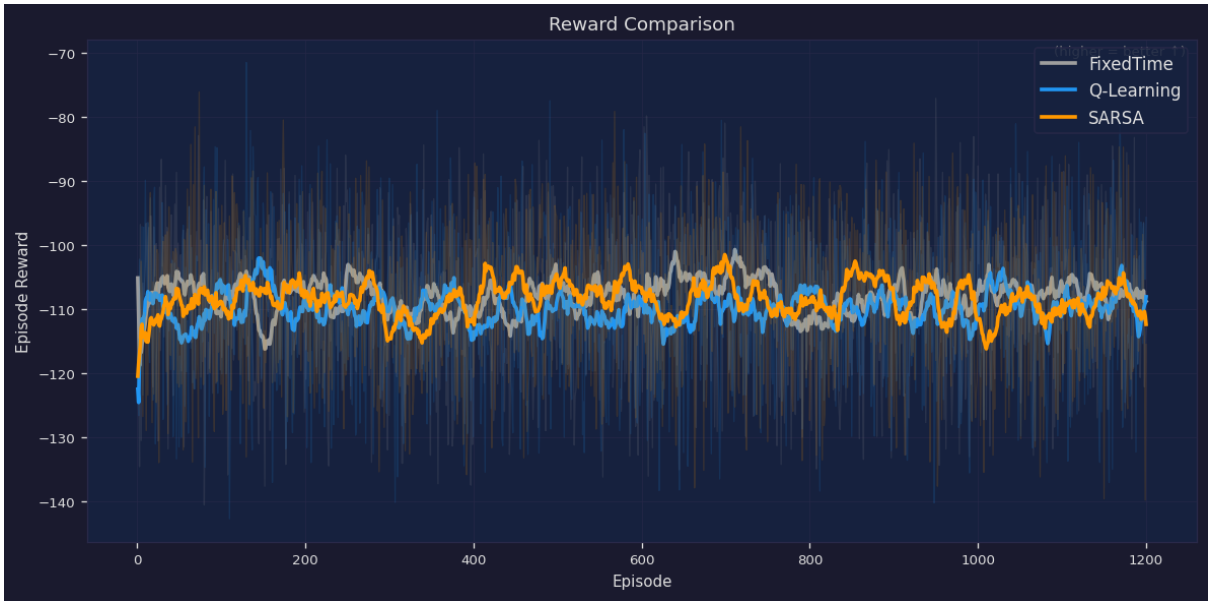

Figure 15: Reward Comparison

Observation: SARSA shows stable reward trends, while Q-Learning fluctuates more.

- Q-Learning's reward instability reflects its aggressive maximization strategy.
- Fixed-Time consistently yields lower reward due to inefficient signal allocation.

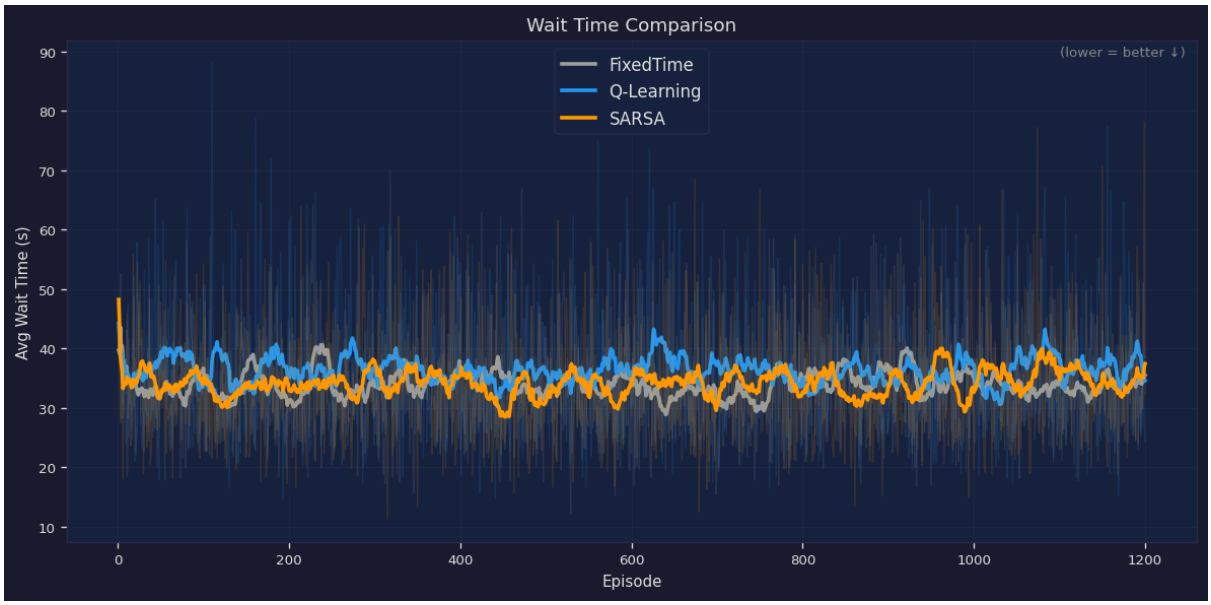

Figure 16: Wait Time Comparison

Observation: SARSA achieves lower average waiting time.

- Q-Learning reduces wait time but with higher variability across episodes.
- Fixed-Time results in higher delays due to lack of real-time adaptation.

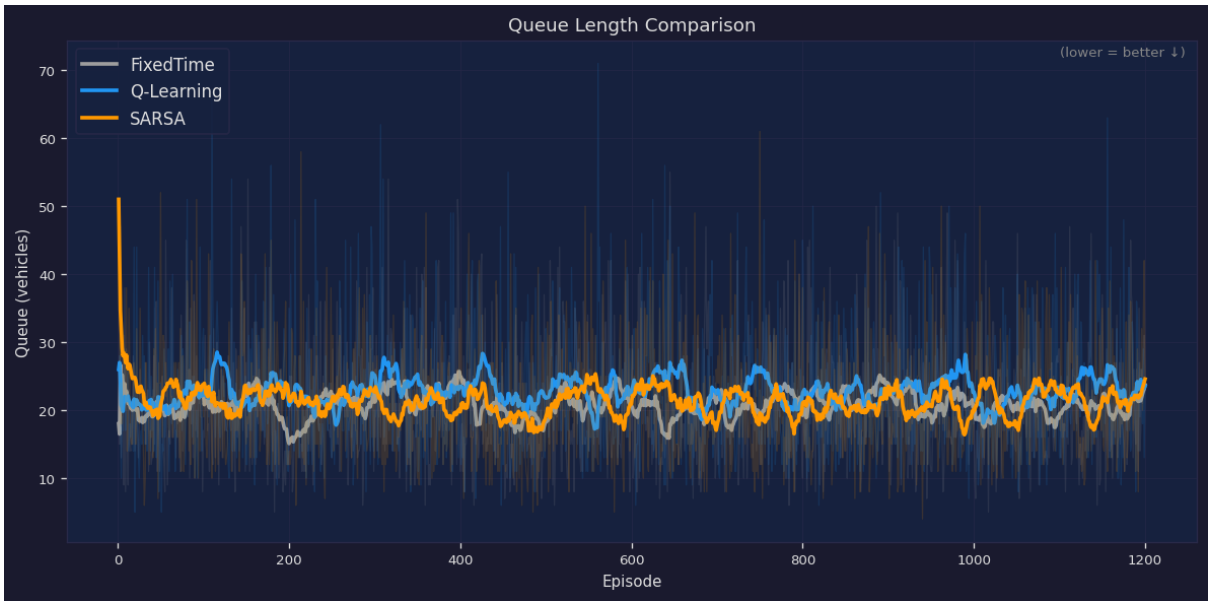

Figure 17: Queue Length Comparison

Observation: SARSA maintains lower and more stable queue lengths.

- Q-Learning shows moderate queue reduction but occasional spikes under dynamic conditions.
- Fixed-Time exhibits larger and persistent queues, especially under uneven traffic distribution.

7 Analysis, Advantages and Disadvantages

7.1 Overall Analysis

The results clearly show that Reinforcement Learning–based traffic control significantly outperforms traditional Fixed-Time systems. Among all approaches, SARSA consistently delivers the best performance in terms of stability, fairness, and overall efficiency.

SARSA achieves lower waiting times, reduced queue lengths, and higher throughput due to its on-policy learning, which enables stable and reliable decision-making under dynamic traffic conditions. In contrast, Q-Learning, being off-policy, shows higher variability and less stable performance.

The Fixed-Time method performs the weakest as it cannot adapt to real-time traffic changes, leading to inefficient signal allocation and increased congestion.

The Jain Fairness Index further confirms that RL-based methods maintain balanced traffic flow, with SARSA showing better consistency. Overall, these results highlight the effectiveness of model-free Reinforcement Learning for adaptive traffic control in complex environments.

7.2 Advantages

- **Adaptive Decision Making:** The system dynamically adjusts signal timings based on real-time traffic conditions, unlike static approaches.
- **Improved Traffic Efficiency:** Significant reduction in waiting time and queue length, along with increased throughput.
- **Fair Signal Allocation:** Maintains balanced traffic flow across all directions, as supported by Jain Fairness Index results.
- **Model-Free Learning:** Does not require prior knowledge of traffic transition probabilities, making it flexible and scalable.
- **Real-Time Applicability:** The system can be extended to real-world scenarios with appropriate sensor integration.

7.3 Disadvantages

- **Simulation Limitations:** The model is tested in a simplified environment without real-world complexities such as pedestrian crossings or multi-lane interactions.
- **Single Intersection Constraint:** The system is limited to one intersection and does not consider network-level traffic coordination.
- **No Real Sensor Data:** Traffic input is randomly generated rather than obtained from real-time sensors.
- **Training Overhead:** RL models require multiple training episodes to converge, which may be computationally expensive.
- **Hyperparameter Sensitivity:** Performance depends on careful tuning of parameters such as learning rate and exploration rate.

8 Performance Comparison

The performance of Fixed-Time, Q-Learning, and SARSA algorithms was evaluated using average reward, throughput, waiting time, and stability obtained from testing and comparison logs.

Model	Avg Reward	Avg Throughput	Avg Wait (s)	Stability
Fixed-Time	-124.75	100 to 120	43.78	Low
Q-Learning	-108.11	90 to 120	38.83	Medium
SARSA	**-104.26**	**100 to 135**	**33.73**	**High**

Table 1: Performance Comparison of Traffic Control Methods

Key Observations:

- **Best Overall Model – SARSA:**
  - Achieves highest Average reward (-104.26), indicating optimal decision-making.
  - Lowest Average waiting time (33.73s) and highest Average throughput (100–130), ensuring efficient traffic flow.
  - High stability due to on-policy learning, making it reliable in dynamic environments.
- **Q-Learning Performance:**
  - Provides moderate improvement over Fixed-Time with better Average reward -108.11 and reduced waiting time.
  - Shows fluctuations during execution due to off-policy updates, resulting in medium stability.
  - Performs well but less consistent compared to SARSA.
- **Fixed-Time Performance:**
  - Records lowest Average reward -124.75 and highest Average waiting time 43.78s.
  - Cannot adapt to changing traffic conditions, leading to inefficient signal allocation.
  - Maintains consistent but limited throughput due to static behavior.
- **Overall Comparison:**
  - Reinforcement Learning methods significantly outperform Fixed-Time control.
  - SARSA provides the best balance between efficiency, stability, and fairness.
  - Q-Learning is competitive but less stable compared to SARSA.

9 Setup and Execution Guide

9.1 System Setup

The project is implemented using Python with a client-server architecture. The backend handles the Reinforcement Learning logic, while the frontend provides a real-time simulation interface.

Requirements:

- Python 3.11.0
- Required libraries: numpy, matplotlib, asyncio, websockets,json
- Web browser (for frontend visualization)

Installation Steps:

- Clone or download the project repository from GitHub:
  - **GitHub Repository Link**
- Navigate to the backend directory:
  - cd Backend
- Install required dependencies:
  - pip install numpy matplotlib websockets

9.2 Running the Project

The system supports three primary modes: Training, Testing, and Comparison.

9.2.1 Training Mode

Training mode is used to train the RL models (Q-Learning and SARSA) over multiple episodes.

```
python server.py --mode train --algo qlearning --episode 1200
```

Options:

- `--algo qlearning`
- `--algo sarsa`
- `--algo fixedtime`

Description:

- Runs multiple episodes
- Updates Q-values based on reward feedback
- Saves trained models in the "models/" directory
- Generates logs including reward, wait time, queue length, and fairness

9.2.2 Testing Mode

Testing mode evaluates a trained model without exploration.

```
python server.py --mode test --algo qlearning --episode 50
```

Description:

- Loads pre-trained model from `models/`
- Runs evaluation episodes

9.2.3 Comparison Mode

Comparison mode runs all algorithms together for performance evaluation.

```
python server.py --mode compare --episodes 1200
```

Description:

- Executes Fixed-Time, Q-Learning, and SARSA sequentially
- Generates comparative graphs
- Stores outputs in `outputs/compare/`

9.3 Frontend Visualization

```
python -m http.server 5000
```

- Start the backend server (Testing mode with any model)
- Open the frontend in a browser
- The frontend connects via WebSocket:
  - http://localhost:5000/
- Displays real-time traffic movement and metrics

9.4 Output Files

- `models/` → Saved trained models
- `outputs/` → Logs and graphs

10 Conclusion and Future Scope

This project demonstrates the effectiveness of Reinforcement Learning for adaptive traffic signal control at a four-way intersection. By formulating the problem as a Markov Decision Process, the system learns optimal signal decisions through continuous interaction with a simulated traffic environment.

The results show a clear improvement over traditional Fixed-Time control. Among the implemented methods, SARSA achieves the best performance, with lower waiting times, higher throughput, and more stable behavior. Q-Learning performs competitively but shows higher variability due to its off-policy nature.

The system successfully adapts to dynamic traffic conditions, reduces congestion, and maintains fairness across all directions. Overall, the project highlights the potential of data-driven approaches in traffic management and demonstrates the practical applicability of Reinforcement Learning to real-world optimization problems.

Future Scope

Although the current system performs well in a controlled environment, several enhancements can be made to extend its applicability to real-world scenarios:

- **Multi-Intersection Control:** Extend the model to handle multiple interconnected intersections for network-level traffic optimization.
- **Integration with Real-Time Data:** Use real sensor data (CCTV, IoT devices) for more realistic and practical deployment.
- **Deep Reinforcement Learning:** Replace tabular methods with Deep Q-Networks (DQN) to handle larger and more complex state spaces.
- **Incorporation of Pedestrians:** Enhance the model to prioritize pedestrian crossings.
- **Adaptive Reward Design:** Further refine reward functions to capture more complex traffic objectives.

These improvements can make the system more robust, scalable, and suitable for deployment in real-world intelligent transportation systems.